

NAVIO: EV TRIP PLANNER PLATFORM
(THAILAND FOCUSED)

MR. SURIYA UPARIPHUTTHIPHONG
MR. PRAPANGKORN THANGSATHITYANGKUL
MR. SUPAKRIT DUANGSRI

A SENIOR PROJECT SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
BACHELOR OF SCIENCE
(COMPUTER SCIENCE)

SCHOOL OF INFORMATION TECHNOLOGY
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY THONBURI

2026

NAVIO: EV TRIP PLANNER PLATFORM
(THAILAND FOCUSED)

Mr. Suriya Upariphutthiphong 66130500858
Mr. Prapangkorn Thangsathityangkul 66130500848
Mr. Supakrit Duangsri 66130500856

B.Sc. (Computer Science)

A Senior Project Submitted in Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science (Computer Science)
School of Information Technology
King Mongkut's University of Technology Thonburi
2026

Project Committee

……. Member and Project Advisor
(Aj. Ekapong Jungcharoensukying)

……. Member
(Asst. Prof. Dr. Chakarida Nukoolkit, Ph.D.)

……. Member
(Assoc. Prof. Dr. Jonathan Hoyin Chan, Ph.D.)

……. Member
(Assoc. Prof. Dr. Vajirasak Vanijja, Ph.D.)

……. Member
(Dr. Vithida Chongsuphajaisiddhi, Ph.D.)

Project Title	Navio: EV Trip Planner Platform (Thailand Focused)
Authors	Suriya Upariphutthiphong (66130500858)
	Prapangkorn Thangsathityangkul (66130500848)
	Supakrit Duangsri (66130500856)
Project Advisor	Aj. Ekapong Jungcharoensukying
Program	Bachelor of Science (Computer Science)
Faculty	School of Information Technology
Academic Year	2026

ABSTRACT

Navio is a browser-based trip planner that combines structured itinerary organisation with electric-vehicle (EV) charging considerations, public plan discovery and community discussions. The project addresses the coordination required when routes, activities, vehicle information and travel advice are handled in separate tools. Its design uses a Next.js client, four Spring Boot domain services, Keycloak authentication and PostgreSQL/PostGIS persistence.

The implemented workflows include private itinerary state, place and route integration, vehicle selection, connector filtering, canonical battery-state calculations and saved-trip optimisation, sanitised public snapshots, Explore, independent-copy APIs and community groups, posts, comments and votes. The original Chapter 3 ER diagrams are retained as target designs and compared with implemented persistence. During report preparation, all 119 frontend checks passed against the latest development snapshot. Earlier backend artifacts and newer Phase 4A verification records are reported with their dates and evidence limits.

The available evidence demonstrates implemented and tested software behaviours, while live two-account acceptance, user evaluation, load capacity and physical battery-prediction accuracy remain outstanding. Full charger feedback persistence, notifications and AI assistance are identified as further development. The report presents the project through Chapter 5 without treating proposed features or expected benefits as measured results.

Keywords: Electric Vehicle / Trip Planning / Charging Estimates / Shared Itinerary / Community / Microservices

ACKNOWLEDGEMENTS

The authors thank Aj. Ekapong Jungcharoensukying for guidance on the Navio senior project and the project committee and School of Information Technology, King Mongkut's University of Technology Thonburi, for their review and academic support.

The authors also acknowledge the official technical documentation and open-source projects used in developing the application. The report builds on the team's earlier chapter documents and design figures and records the evidence and limitations of the current implementation.

CONTENTS

CHAPTER 1 INTRODUCTION 1
1.1 Background Information 1
1.2 Objectives 1
1.3 Scope 2
1.4 Expected Benefits 3
CHAPTER 2 FEASIBILITY STUDY 4
2.1 Introduction 4
2.2 Problem Statement 4
2.3 Related Research and Projects 4
2.4 Requirement Specifications 6
2.5 Implementation Techniques 7
2.6 Implementation Plan 8
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 10
3.1 Introduction 10
3.2 Analysis of the Existing System 10
3.3 User Requirement Analysis 10
3.4 System Design 12
CHAPTER 4 SYSTEM FUNCTIONALITY 24
4.1 Introduction 24
4.2 System Architecture 25
4.3 Test Plan 26
4.4 Test Results 28
CHAPTER 5 SUMMARY AND SUGGESTIONS 31
5.1 Project Summary 31
5.2 Problems Encountered and Solutions 32
5.3 Suggestions for Further Development 34
REFERENCES 36
APPENDIX A TEST AND SOURCE RECORD 38
A.1 Report Basis and Document Provenance 38
A.2 Frontend Test Reproduction 38
A.3 Sample Integrated Acceptance Record 38
A.4 Backend Artifact Interpretation 39
A.5 Version Record 39
A.6 Code Evidence Map 40

LIST OF TABLES

Table 2.1: Related-system roles and the Navio design response 5
Table 2.2: Functional requirements carried into evaluation..... 6
Table 2.3: Development sequence and reviewable outputs 8
Table 3.1: Principal actors and use cases 11
Table 3.2: Target data design compared with inspected persistence 14
Table 3.3: Canonical energy and optimisation contract..... 22
Table 4.1: Current implementation modules and representative interfaces 25
Table 4.2: Functional and access-control acceptance scenarios 26
Table 4.3: Frontend checks rerun on 7 October 2026 against ba01a1b 28
Table 4.4: Retained backend test artifacts and their dates 29
Table 5.1: Objectives, available evidence and remaining work 31
Table A.1: Acceptance record form for the publication and copy scenario..... 38
Table A.2: Representative source locations for report claims 40

LIST OF FIGURES

Figure 3.1: Main Navio navigation and access groups 12
Figure 3.2: Current implemented service boundaries and supporting infrastructure 13
Figure 3.3: Private planning, controlled publication and independent-copy data flow .. 14
Figure 3.4: Original target trip ER diagram from the Chapter 3 report 16
Figure 3.5: Original target EV ER diagram from the Chapter 3 report 17
Figure 3.6: Original target user and IAM ER diagram from the Chapter 3 report 18
Figure 3.7: Original target notification ER diagram from the Chapter 3 report 19
Figure 3.8: Original target community ER diagram from the Chapter 3 report 20
Figure 3.9: Original target AI ER diagram from the Chapter 3 report 21

LIST OF SYMBOLS

Symbol	Meaning	Unit
B	Known usable battery capacity	kWh
c	Battery-side energy consumption	kWh/100 km
d	Road distance	km
E	Nominal energy	kWh
P	Effective charging power	kW
R	Rated range	km

LIST OF TECHNICAL VOCABULARY AND ABBREVIATIONS

Abbreviation	**Meaning**
API	Application Programming Interface
BFF	Backend for Frontend
CRUD	Create, Read, Update and Delete
DC / AC	Direct Current / Alternating Current
DTO	Data Transfer Object
ERD	Entity–Relationship Diagram
EV	Electric Vehicle
JWT	JSON Web Token
OIDC	OpenID Connect
SoC	State of Charge
SSR	Server-Side Rendering
UX / UI	User Experience / User Interface

CHAPTER 1
INTRODUCTION

1.1 Background Information

Planning a road trip requires travellers to coordinate destinations, transport, accommodation and activities. An electric vehicle (EV) introduces further decisions about available battery energy, compatible connectors, charging locations and the time needed to recharge. The International Energy Agency's Global EV Outlook 2025 discusses vehicle deployment, batteries and charging infrastructure as related parts of electric mobility [1]. For an individual journey, these considerations must be interpreted alongside the itinerary rather than treated as an isolated station search.

The team's earlier Navio report identified a fragmented planning workflow: a traveller may use a mapping application, an itinerary organiser, a charging application and travel discussions to prepare one trip [2]. Information collected in one tool must often be transferred to another. Changing the order of attractions can alter both the driving route and charging needs, while changing the vehicle can alter which stations are suitable. This project addresses that coordination problem through a browser-based planning workspace.

Route distance and duration are useful inputs, but they do not establish that an EV itinerary will be practical under every operating condition. Google Routes provides route computation, while charging compatibility and battery estimates require additional application logic [3]. A listed station also does not guarantee an available connector when a traveller arrives. Navio therefore presents route and charging information as planning information and does not claim to reserve chargers or verify physical availability.

Navio: EV Trip Planner Platform (Thailand Focused) combines day-by-day itinerary management, EV-related planning, public plan discovery and community discussions. Thailand remains the primary project context. The location model can also represent destinations elsewhere when the provider supplies relevant data. This latest report extends the team's Chapter 3 analysis and design through system functionality and project conclusions, using the supplied capstone outline and thesis formatting guidance [4,5].

1.2 Objectives

The primary objective is to design and develop a web application that brings structured trip planning and EV charging considerations into a single workflow. The specific objectives carried forward from the project design are as follows.

- Develop a trip planner that supports creating, editing, organising and reusing multi-stop itineraries, including day schedules, notes, checklists and travel expenses.
- Integrate place discovery, route visualisation, vehicle information, connector compatibility and charging-stop planning with the itinerary.
- Allow users to publish selected itinerary information, discover shared plans and create independent personal copies.
- Provide community groups, posts and threaded discussion so travellers can exchange experiences and advice.

- Protect private data through authentication, ownership checks and role-based administration, and assess implemented behaviour through reproducible software tests.
- Consider AI-assisted drafts and itinerary suggestions after the main workflows are stable, while retaining user control over any proposed changes.

The last objective is conditional. The original design included an AI component, but no AI Planning service directory exists in the inspected backend. AI assistance is consequently treated as further development rather than a completed result [6]. Chapter 5 evaluates the objectives against the evidence available for this version.

1.3 Scope

1.3.1 Core Trip Planning

The core planner supports trip creation, date and destination selection, day-by-day itinerary blocks, place stops, notes and checklists. Users can change stop order and inspect routes in a map panel. Budget and expense information supports travel organisation. Signed-in users work with persisted trips, while guest planning provides temporary planning facilities without granting access to a member's private data.

Trip state is managed separately from public publication. Saving an itinerary updates the private working plan; it does not automatically publish every subsequent edit. The current trip representation includes a planner snapshot API, immutable trip vehicle-energy snapshots, an explicitly saved initial state of charge (SoC), optional observed SoC at stops and trip-specific garage membership. The latest development migrations add those energy and membership records without rewriting earlier trip data. Features shown in the target data design, such as revision history and comprehensive collaboration, are not assumed to be complete merely because they appear in an ER diagram.

1.3.2 EV Integration

EV-related planning includes catalogue and custom vehicles, a personal garage, compatible-connector filtering, nearby charger discovery, manual charger insertion and route optimisation logic. The latest development contract separates direct-consumption and rated-range models, uses explicitly saved trip battery state and distinguishes predictions from user observations. Rated-range estimates are provisional and preview-only for automatic charging-stop application. Section 3.4 explains that contract.

The application does not control a charger, make a charging payment, receive continuous vehicle telemetry or guarantee live occupancy. Charging reviews, reports, corrections and verification were included in the earlier EV data design, but the inspected mobility migration does not create the full charger-domain tables. Those functions remain design requirements that need persistence and workflow implementation.

1.3.3 Explore and Plan Sharing

Explore displays published itinerary snapshots rather than private trips. An owner can manage publication, select supported inclusion options, control listing in Explore and stop sharing. Public readers can view a shared itinerary; a signed-in reader can request

an independent copy. A custom public title and author byline are separate from private account identifiers. View-count metadata supports trending discovery.

Independent copying protects the original plan from edits by the receiving user. Public publication is also distinct from simultaneous collaborative editing of one trip. This report describes the implemented publication and copy contract and retains broader collaborative features as further work where acceptance evidence is unavailable.

1.3.4 Community and Supporting Functions

Community functions include group discovery and membership, posts, threaded comments, voting and supported image uploads. Authentication, profiles, saved vehicles, a managed global vehicle catalogue and administration support the main workflows. Group moderation and account administration are distinct permissions; a global role does not automatically establish ownership of another user's trip.

The earlier design proposed notifications, media records, reporting and optional AI data. Some of these structures remain target designs. Email, SMS and push delivery, a complete notification experience, autonomous AI itinerary changes and physical charger booking are outside the completed implementation assessed here.

1.3.5 Platform and Evaluation Boundaries

The deliverable is a web application rather than a native mobile application. It is intended for an academic demonstration and small deployment. Earlier concurrency figures are treated as planning targets because no documented load test establishing the achieved capacity was found. The report does not claim experimentally validated battery accuracy, a globally optimal route, reduced travel anxiety or measured financial savings.

1.4 Expected Benefits

The expected user benefit is a more coherent planning workflow: activities, route information and charging considerations can be reviewed together. Copying a public itinerary can reduce repeated data entry while allowing a traveller to customise dates and preferences. Community discussion can add practical context to a plan, provided user contributions remain distinguishable from independently verified data.

For the development team, the project demonstrates integration of a stateful web interface, independently owned service APIs, relational data, external geospatial providers and access control. These are expected benefits of the design and implementation. Their effect on task completion time, user understanding and real travel outcomes requires the further evaluation proposed in Chapter 4.

CHAPTER 2
FEASIBILITY STUDY

2.1 Introduction

This chapter refines the original problem statement, reviews related systems, defines requirements and explains the implementation approach. It carries forward the team's feasibility study while updating technology choices to match the inspected project. Technical feasibility concerns whether the system can maintain data and compute consistent results. Practical effectiveness concerns whether travellers understand those results and find the workflow useful; that requires user and route-based evaluation beyond unit tests.

2.2 Problem Statement

2.2.1 Fragmented Planning Information

The earlier project analysis described travellers combining navigation, itinerary planning, charger discovery and informal advice [2]. The resulting coordination work includes copying destinations, comparing connector types and revisiting charging choices after itinerary changes. Navio's design response is to make the itinerary the organising structure and associate route and EV information with its stops and days.

2.2.2 Uncertainty About Charging Conditions

Location, connector type and advertised power are useful station attributes, but the usable charging experience can depend on conditions that a planning application does not directly observe. Provider data may be incomplete or stale. A user contribution can help identify an issue but is not a substitute for operational telemetry. The application must make missing or estimated information apparent and avoid presenting an unverified station listing as a guarantee.

2.2.3 Reuse, Privacy and Maintenance

Unstructured travel advice is difficult to reuse as an editable itinerary. Directly exposing a private trip would also reveal information the owner may not intend to share. The project therefore separates private persistence, sanitised publication and copying. A further engineering problem is keeping client, gateway, service and database contracts compatible as the application develops.

2.3 Related Research and Projects

2.3.1 Electric Mobility and Charging Infrastructure

The IEA report provides background on the relationship between EV deployment and charging infrastructure [1]. It supports the importance of considering charging in travel planning, but it does not validate Navio's algorithm or prove the requirements of a particular user group. The application-specific requirements are design inferences, and the report does not claim that a formal user survey was conducted.

2.3.2 Mapping and Route Services

Google Routes is an external route-computation service used to obtain driving geometry, distance and duration [3]. Google Places supplies place search and place information [7]. These capabilities reduce the need to maintain a complete destination and road database locally. Navio adds application state around those responses: itinerary days, user notes, vehicle parameters, publication choices and ownership rules.

The comparison concerns the workflow Navio implements. It does not assert that every version of Google Maps lacks EV functions. Features and regional coverage of external products can change; provider documentation is a more reliable basis than an absolute claim about product limitations.

2.3.3 Itinerary Planning Platforms

Wanderlog presents itinerary organisation, shared guides, bookings and a combined itinerary-and-map view [8]. This makes it a relevant reference for organising a multi-day trip. Navio's particular project focus is connecting that organisation with the selected EV, charging choices and a separately published itinerary snapshot. No controlled usability comparison or comprehensive feature audit of Wanderlog was performed.

2.3.4 Charger Applications and Travel Communities

The original Chapter 3 considered Saifah, PUGEV, travel blogs and community forums as examples of charging or experience-oriented workflows [2]. They informed the distinction between finding a station and organising a complete trip. The supplied project materials do not include a dated, reproducible benchmark of those applications, so this report does not repeat categorical claims that they cannot support particular features.

Travel discussions provide context such as destination advice or charging experiences, but their quality and currency vary. Navio's community design retains structured groups and threads and can reference a travel plan. It does not treat a vote, comment or user report as independently verified charger availability.

Table 2.1: Related-system roles and the Navio design response

System category	Documented or analysed role	Navio design response
Mapping services [3,7]	Place information and road-route computation	Associate provider results with editable itinerary days and stops.
Itinerary platforms [8]	Organising destinations, bookings and shared guides	Combine itinerary organisation with vehicle and charging considerations.
Charger-oriented workflows [2]	Finding and inspecting charging information	Let selected chargers participate in an itinerary and energy estimate.
Travel communities [2]	Exchanging experience and destination advice	Keep structured discussions alongside reusable plans without claiming telemetry.

2.4 Requirement Specifications

2.4.1 User Roles

Visitors may browse public plans and permitted community information. Members may save private trips and vehicles and contribute through authenticated workflows. Moderators and administrators have additional actions within the implemented permission model. OWNER is a privileged global role used for controlled administrator management; assigning it remains an identity-operator task. Trip ownership is enforced by the Trip Planning service rather than inferred from a visible button.

2.4.2 Functional Requirements

Table 2.2: Functional requirements carried into evaluation

ID	Requirement	Acceptance criterion
FR-01	Account authentication and session handling	Private account operations require a valid session; public reads remain available.
FR-02	Trip and itinerary persistence	An owner can save dates, stops and ordering and recover the saved state.
FR-03	Place and route integration	Valid stops produce provider-backed locations and route information; unavailable responses show an error.
FR-04	Vehicles and charger compatibility	Vehicle inputs persist and compatibility filtering follows the selected vehicle.
FR-05	EV planning estimates	Energy and charging calculations use valid inputs and an explicit versioned policy.
FR-06	Publication and independent copying	Selected public data is separated from private edits; copying does not change the source trip.
FR-07	Community participation	Permitted users can use groups, posts, comments and votes under the service's rules.
FR-08	Account and catalogue administration	Privileged operations require the appropriate role and preserve an audit reason where required.

ID	Requirement	Acceptance criterion
FR-09	Conditional AI assistance	Any later AI proposal uses validated data and requires user confirmation before application.

2.4.3 Non-functional Requirements

Security requires server-side ownership enforcement, validated requests and controlled forwarding of user identity. Maintainability requires explicit service ownership and versioned migrations. Reliability requires safe error handling when a provider times out or a saved request fails. Usability requires keyboard access, clear loading and error states and a responsive planner layout.

Performance is constrained by external APIs and the deployment resources. Caching can reduce repeated requests, but no measured latency or capacity is claimed here. Database compatibility requires additive changes and startup validation against PostgreSQL. H2-backed tests and controller slices cannot alone establish compatibility with PostgreSQL/PostGIS.

2.5 Implementation Techniques

2.5.1 Frontend and State Management

The inspected client uses Next.js App Router, TypeScript and React. Next.js documents the App Router model and its application conventions [9]. Jotai manages client interface state, while TanStack Query manages API data and mutations. The planner combines reusable interface components rather than maintaining a separate set of cards for every screen. Auth.js integrates the web session with Keycloak.

The client package declares Next.js 16.3.4 and React 19.2.3. These are project-manifest values, not claims about the latest public releases. Zod validates API shapes, and the application has server-side route handlers that proxy selected backend routes. Provider credentials and privileged service calls belong on the server side.

2.5.2 Backend and Authentication

The inspected backend uses Java 25 and Spring Boot 4.1.1. Spring Boot supplies the application framework and its standard integration facilities [10]. Four domain services are present: User Management, Trip Planning, Mobility and EV, and Community. Gateway, configuration and discovery are separate platform applications.

Keycloak provides credentials, identity sessions and OpenID Connect tokens [11]. The User Management service stores Navio profiles and application information; it does not replace Keycloak as a password authority. Spring Cloud Gateway validates tokens and forwards trusted identity context to private downstream services. This current arrangement supersedes Clerk and the combined IAM-and-trip service shown in the earlier design.

2.5.3 Persistence and Integration

PostgreSQL stores relational data in service-owned schemas. Schemas provide a logical namespace within a database [12]; the deployment inspected here does not require a separate database instance for each service. PostGIS provides spatial database capabilities [13]. Flyway versioned migrations preserve an ordered schema history [14].

Kafka is included for asynchronous integration [15], but a broker being deployed does not establish that every proposed event workflow has been completed. Media storage and observability are supporting infrastructure. The production configuration inspected here includes Prometheus, Grafana, Loki and Zipkin; an earlier architecture document describes a different hosted-observability target. The report follows the inspected configuration when describing the current deployment.

2.5.4 EV Calculation and Provider Adapters

The project uses provider adapters for external places and road routes. The latest development energy model, navio-energy-v1, calculates nominal consumption without silently adding an arbitrary correction factor. It distinguishes direct consumption, rated-range fallback and unavailable information. Usable capacity is not inferred from a declared gross capacity, and uncertainty remains explicitly uncalibrated.

Optimisation is bound to saved trip state. The client waits for autosave, checks the saved version and requires the canonical-optimization-v1 service capability. The trip service supplies saved vehicle provenance, explicitly saved initial SoC and preceding itinerary context to mobility. Rated-range estimates may be previewed but cannot authorise automatic charging-stop application. Further all-leg feasibility and preview/apply precision checks remain engineering work packages, not completed claims.

2.6 Implementation Plan

The team's earlier report divided implementation into architecture and interface design, core planning, EV integration, sharing, community integration, conditional AI work, integrated evaluation and finalisation [2]. The sequence remains useful, but it is a proposed work plan rather than a retrospectively verified schedule. No approved milestone calendar or time-sheet dataset was supplied, so this version does not invent completion dates or actual person-hours.

Table 2.3: Development sequence and reviewable outputs

Stage	Main work	Reviewable output
1	Requirements, service boundaries, UX and database design	Scope, workflows, prototypes and target ER diagrams.
2	Authentication, base planner and persistence	Saved private trips and protected account APIs.
3	Vehicles, map providers and EV logic	Garage, charger discovery and tested calculation policy.

Stage	Main work	Reviewable output
4	Publication and Explore	Sanitised snapshot, listing and independent-copy contract.
5	Community and administration	Groups, posts, comments, permissions and catalogue management.
6	Conditional AI extension	Validated editable proposals if the main workflow is stable.
7	Integrated testing and evaluation	Scenario evidence, PostgreSQL checks and user acceptance records.
8	Finalisation and deployment	Version-compatible release and updated report.

The latest report prioritises completing integrated acceptance and documenting limitations. Advanced additions should follow a stable, tested base rather than changing the principal data contract during final evaluation.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter extends the team's latest Chapter 3 analysis [2]. The earlier report supplies the project problem, principal workflows and six ER diagrams retained here. The architecture and data-flow descriptions are updated to the inspected implementation. Target-design diagrams remain useful design records, but they are explicitly distinguished from the tables and APIs currently present.

3.2 Analysis of the Existing System

3.2.1 Existing Planning Workflow

A traveller first chooses destinations and approximate dates, collects attractions and accommodation options, orders the destinations, and examines driving routes. An EV traveller additionally selects suitable charging opportunities and checks that the proposed journey is compatible with the vehicle. Advice from blogs or discussions may cause further changes. The earlier project analysis observed that these tasks often occur in separate tools [2].

The difficulty is the relationship between the tasks. A charger selected before the route is finalised may create a large detour. Moving an activity to another day can change the day's end location. An itinerary copied from another traveller may use a vehicle with different connectors. Navio's design response is to retain structured planning state and recompute relevant information when that state changes.

3.2.2 Mapping, Itinerary and Charging Views

The original comparison considered mapping applications, Wanderlog, charger applications and travel communities. The updated analysis focuses on each tool's role rather than making unverified claims about its current feature set. Mapping provides route and place data; itinerary applications organise activities; charging tools provide station information; communities supply experience and discussion.

Navio combines these roles within the trip workspace and adds persistence and publication controls. The application still depends on external providers for important information. Integration reduces the amount of manual coordination the design requires, but no user study has yet measured the reduction.

3.3 User Requirement Analysis

3.3.1 Basis of the Requirements

Requirements were derived from the supplied project report, comparison of existing workflows, project discussions and the implementation's intended tasks. The source report refers to user reviews and online discussions, but does not supply a questionnaire, participant count, interview transcript or coded response dataset. The requirements in this report are therefore presented as design requirements; they are not reported as statistical survey findings.

The principal requirements are an integrated itinerary-and-map view, understandable EV estimates, reusable shared plans, community discussion and clear privacy boundaries. Different users also need different access: a guest exploring a destination, a member managing a private trip and a privileged administrator should not receive equivalent write permissions.

3.3.2 Principal Use Cases

Table 3.1: Principal actors and use cases

Actor	Use case	Design condition
Guest	Browse Explore and shared itineraries; try temporary planning	Public data only; persistent private writes require sign-in.
Trip owner	Create and edit a trip; choose stops, vehicle and charging settings	The service validates ownership for private resources.
Public reader	View a published plan	Serve a sanitised frozen snapshot rather than the private working trip.
Signed-in reader	Copy a shared plan	Allocate a separate private trip; preserve source integrity.
Community member	Join groups and participate in discussions	Validate the group's participation rules and content ownership.
Moderator / administrator	Perform permitted moderation or catalogue actions	Role checks and audit information apply to privileged operations.

3.3.3 Information and Interaction Requirements

The planner should retain itinerary context while the user searches for a place or charger. Changes should have clear save, loading and error feedback. Missing provider data must remain visibly missing rather than replaced by a fabricated distance or power. Charging duration should be recognisable as an estimate. Public viewing should use the same itinerary concepts as private planning, with editing controls disabled.

The design also requires distinguishing a reusable vehicle definition from the state of a particular journey. Starting battery and a plan's charging choices are contextual information; changing an account vehicle should not be interpreted as a direct observation of the battery in an ongoing trip. The latest development implementation separates account definitions from trip membership and immutable energy snapshots; starting SoC and stop observations belong to the trip. It does not provide continuous physical vehicle telemetry.

3.4 System Design

3.4.1 Site Map

The site map separates public browsing and guest planning from persisted member operations. Administration is an additional role-gated route family. This fills the site-map placeholder in the earlier Chapter 3 report. It represents the main navigation relationships; provider pages and identity-provider screens are external to this map.

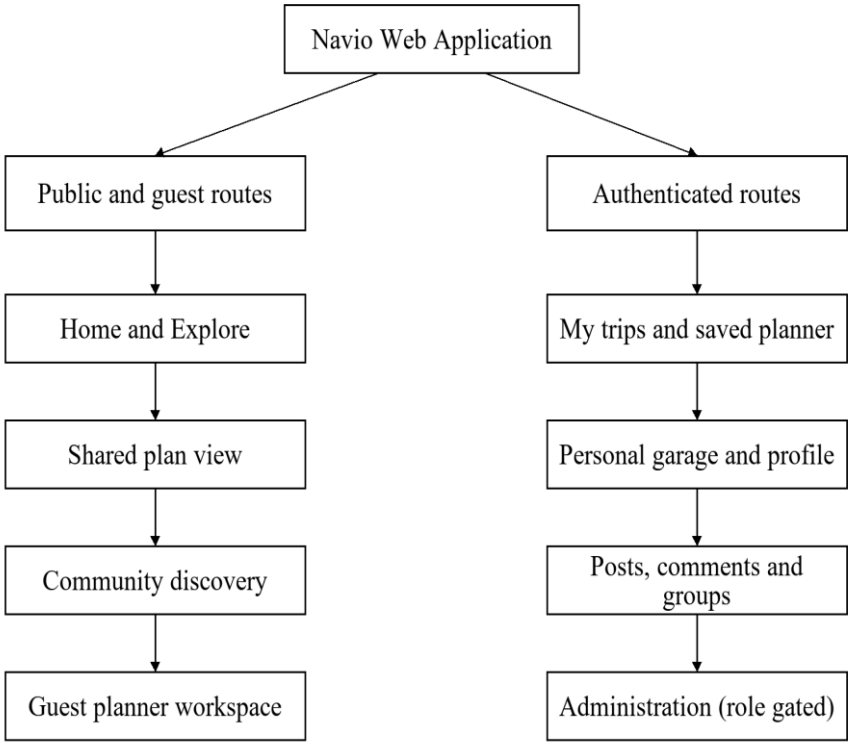


Figure 3.1: Main Navio navigation and access groups

3.4.2 Frontend and Backend Architecture

The Next.js client presents the interface and selected backend-for-frontend (BFF) routes. Production traffic enters through NGINX. Application API traffic is routed through Spring Cloud Gateway, which integrates with Eureka service discovery. Keycloak provides the identity protocol. Configuration Server supplies service configuration.

The four domain services have distinct responsibilities. User Management owns profiles, saved vehicles, catalogue administration and account audit functions. Trip Planning owns private itinerary state and publication. Mobility and EV owns external places, routes, charger queries and optimisation logic. Community owns groups, posts, comments and associated media workflows. The optional AI design is not included as an implemented domain service.

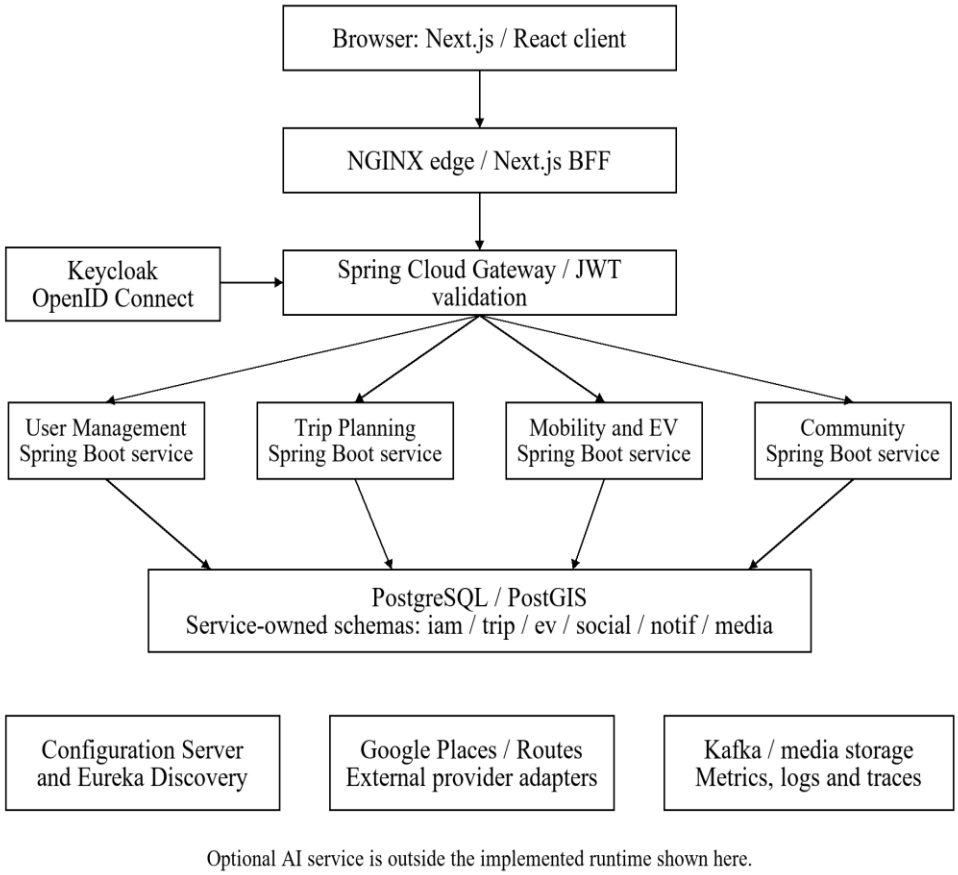


Figure 3.2: Current implemented service boundaries and supporting infrastructure This current architecture supersedes the earlier Clerk, Vault and combined IAM-and-trip representation. PostgreSQL is deployed as a shared instance with logical schema ownership. The report does not imply that the target hosted-monitoring profile, a complete event outbox across all domains, or a separate notification application is already running.

3.4.3 Data Flow and Publication Boundary

An authenticated mutation passes from the client through a BFF route and gateway to the owning service. The service validates input and ownership before changing persisted data. Provider-dependent requests are handled through the Mobility and EV adapters. A failed provider response must be represented as a recoverable failure or unavailable result rather than a successful fabricated plan.

Publication introduces a second data boundary. The owner previews inclusion options, and the server sanitizer constructs a public snapshot. Publishing stores that snapshot, so later private changes remain private until the owner updates publication. A public read returns the frozen representation. Copying reconstructs an independent private trip for the receiving account and requires authentication.

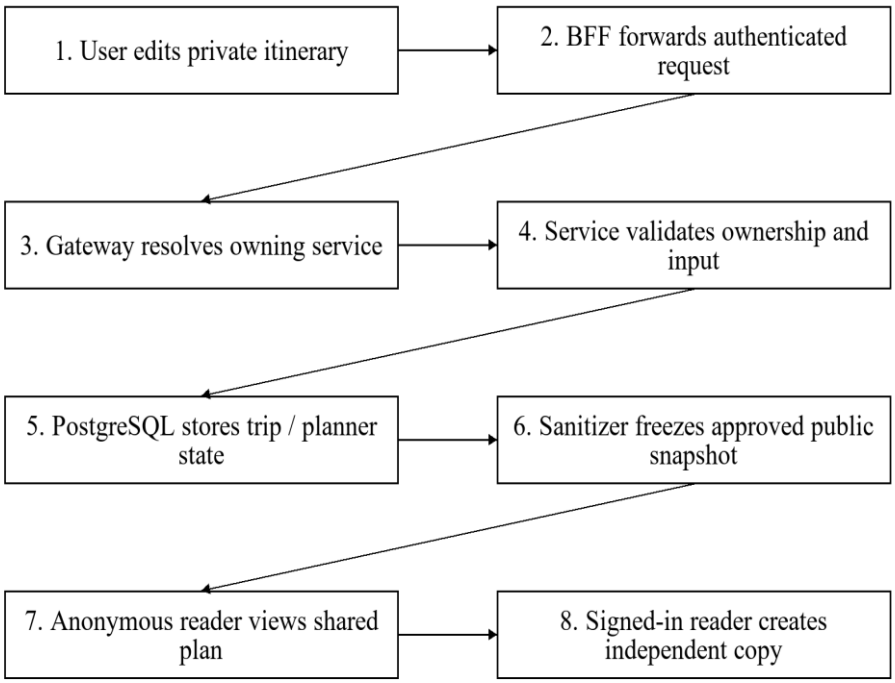


Figure 3.3: Private planning, controlled publication and independent-copy data flow

3.4.4 Data Structures and ER Diagrams

Figures 3.4–3.9 are the team's original ER diagrams copied from the Chapter 3 Word file without redrawing their contents [2]. They document the broader target design. Their plural table names and proposed entities do not always match the current Flyway migrations. Table 3.2 identifies the inspected implementation so the figures are not mistaken for a complete deployed schema.

Table 3.2: Target data design compared with inspected persistence

Domain	Inspected persisted structure	Target-design items still requiring reconciliation
IAM	users, user_roles, user_bans, user_vehicles, saved places, user pictures, vehicle_models and audit/outbox records	Compare the ERD's ACL and generic media entities with current service ownership.
Trip	trip, list_block, block_item, checklist_sub_item, expense, trip_publication and V14/V15 energy/garage additions	Target trip members, revisions, attachments and broader split/collaboration entities are not established by these migrations.
EV	Initial migration enables PostGIS; provider and optimisation code is present	Full charger, review, report, suggestion and cached-tile persistence is not created by the inspected mobility migration.

Domain	**Inspected persisted structure**	**Target-design items still requiring reconciliation**
Community	Groups, profiles, memberships, rules, flairs, resources, posts, comments, votes and group media	Reconcile the wider target moderation, search and notification structures with actual workflows.
Notifications and AI	Target records appear in the earlier ER diagrams	No completed notification delivery experience or AI Planning service is established in the inspected tree.

3.4.4.1 Trip Data Design

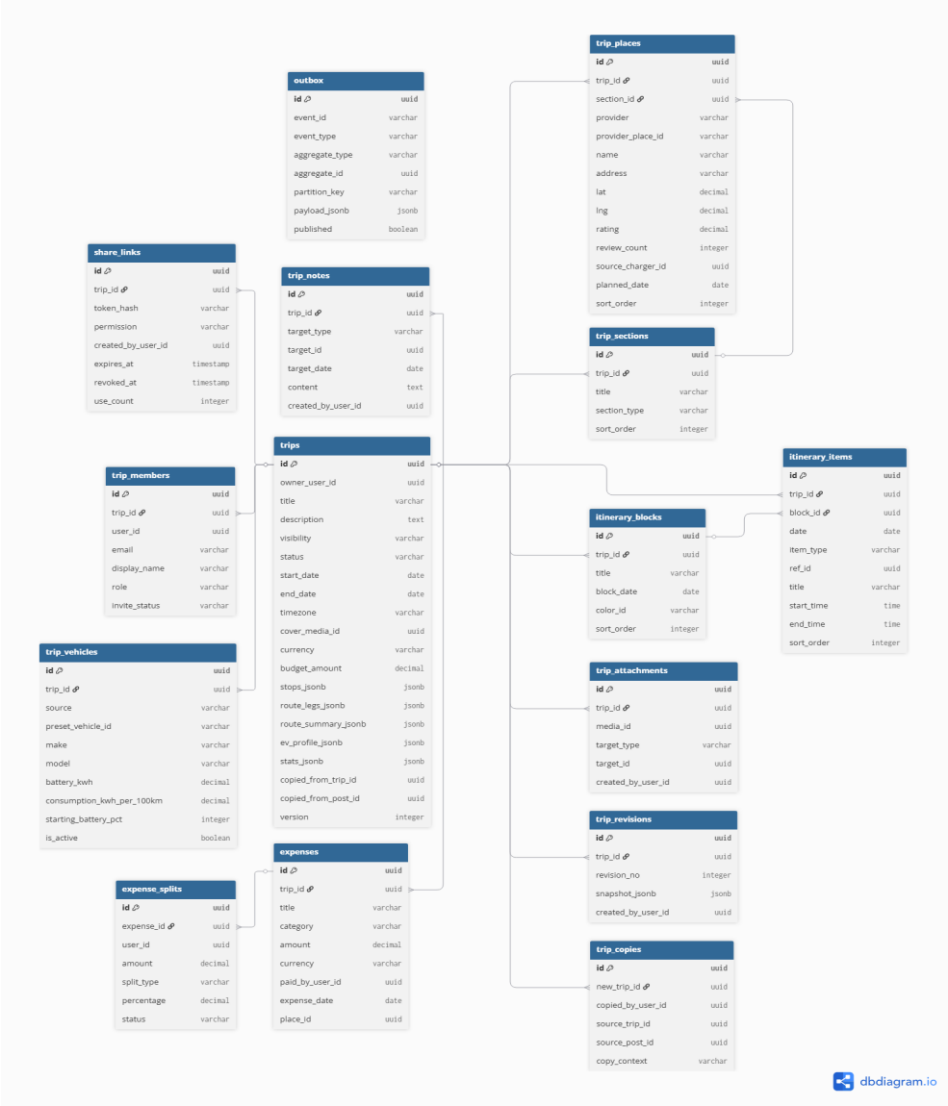


Figure 3.4: Original target trip ER diagram from the Chapter 3 report

The original trip design centres on trips and their itinerary structure. It proposes destinations, vehicle information, expenses, membership, share links, notes, attachments, revision history and integration events. These relationships explain the intended separation between the trip itself and its supporting records. The current implementation instead uses trip.trip with list blocks, block items, a publication snapshot and additive V14/V15 energy-state and trip-garage records. The diagram is retained as a design reference; it is not a claim that all proposed entities are deployed.

3.4.4.2 EV Data Design

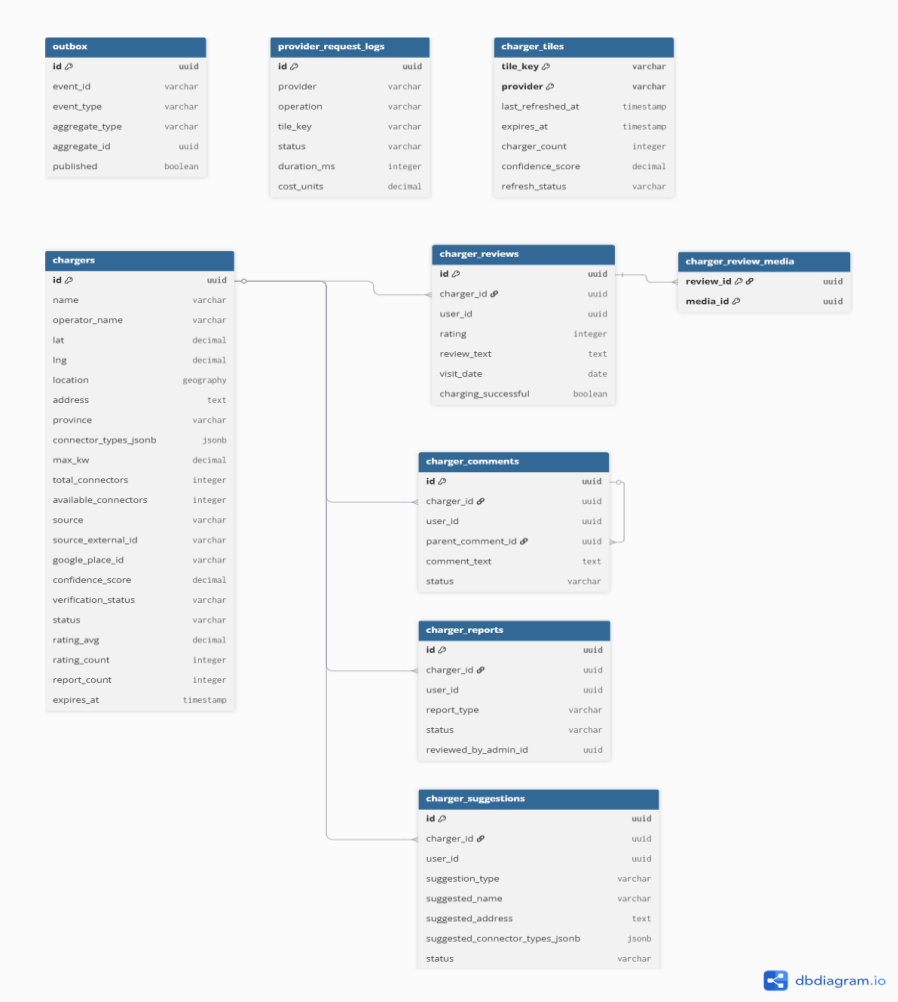


Figure 3.5: Original target EV ER diagram from the Chapter 3 report

The EV design proposes charger metadata, cached tiles, provider-request records, reviews and media, threaded comments, reports, suggestions and outbox events. The intended relationship is that a charger is the central reference for user feedback and later verification. The current service provides provider-backed discovery and optimisation, but its inspected initial migration does not create these full domain tables. Implementing and testing that persistence is a further-development task.

3.4.4.3 User and IAM Data Design



Figure 3.6: Original target user and IAM ER diagram from the Chapter 3 report

The IAM design proposes users, roles, access records, restrictions, media, saved vehicles and audit information. Current migrations implement users, roles, bans, vehicles and further account records in the iam schema. Keycloak remains authoritative for credentials and global identity sessions. The diagram's application-user relationships should therefore be read alongside the current identity boundary rather than as an independent password database.

3.4.4.4 Notification Data Design

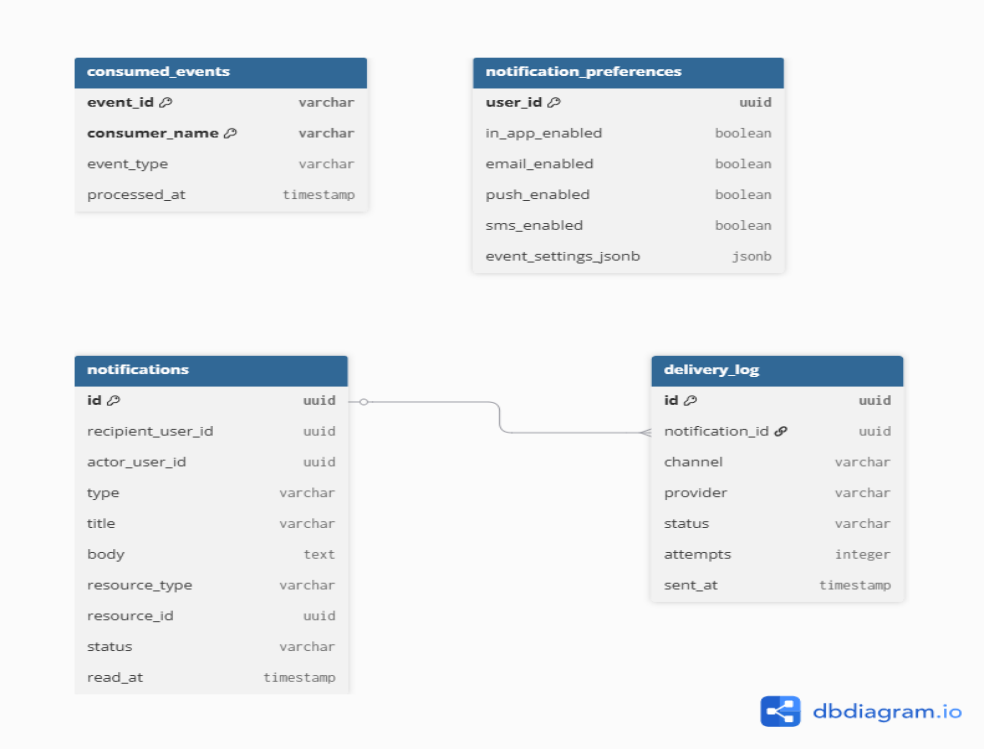


Figure 3.7: Original target notification ER diagram from the Chapter 3 report

The notification design proposes records for notifications, recipients and preferences so events can be presented to the relevant user. It is a target module associated with Community, not evidence of a deployed standalone Notification service. A complete user-facing notification workflow and external delivery channels were not verified for this report. They require explicit event contracts, duplicate handling, privacy rules and acceptance tests.

3.4.4.5 Community Data Design



Figure 3.8: Original target community ER diagram from the Chapter 3 report

The community design relates groups and memberships to posts, comments, interactions and moderation records. Current migrations implement core group, post, comment and vote entities in the social schema and supporting group-media records. Some broader entities in the original diagram remain target structures. The service should enforce community ownership and membership separately from the ownership of a trip referenced in a post.

3.4.4.6 AI Data Design



Figure 3.9: Original target AI ER diagram from the Chapter 3 report

The AI design proposes prompt configurations, sessions, messages, tool invocations, usage logs and quota counters. Those records would support traceable model use and bounded assistance. Since no AI Planning service is present in the inspected backend, this figure is retained as an optional extension design. A future implementation must validate proposals against existing trip and mobility contracts and require user confirmation before changing a plan.

3.4.5 Canonical Energy, Battery State and Optimisation

The latest development version uses navio-energy-v1 in Java CanonicalEnergy and the client energy-model.ts. For a direct-consumption model, nominal energy is $E = d \times c / 100$, where d is road distance in kilometres and c is battery-side consumption in kWh/100 km. If usable battery capacity B is known, nominal SoC use is $100 \times E / B$. The canonical model does not multiply this nominal result by the earlier simulation's 12% planning margin.

For rated range R, provisional SoC use is $100 \times d / R$. Energy can be expressed in kWh only when a suitable capacity is available; otherwise it remains unavailable. Rated range is not used to fabricate a direct consumption figure or infer usable battery capacity. If the required inputs are missing, results include explicit availability and reason fields.

State propagates chronologically across stops and days. Predicted arrival is departure SoC minus nominal use. The raw result preserves a negative arrival as an infeasible leg; the clamped display value is not evidence that the leg is feasible. An observed SoC remains a distinct user input and can establish the subsequent departure state, while the earlier prediction and its feasibility result remain available for interpretation.

Charging uses compatible connectors and effective battery-side power. A target establishes an intended post-charge SoC; with known capacity, the required energy follows from the percentage increase. Nominal charging minutes are 60 × energy / effective power when the inputs are available. Duration-based charging can similarly propagate energy when power and capacity are known. The model does not claim vehicle-specific taper, calibrated efficiency or physical charging accuracy.

The default arrival reserve is 12%, but reserve is separate from nominal energy and is not an additional consumption multiplier. The result reports uncertaintyStatus as UNCALIBRATED with a null uncertainty allowance. This prevents a configured reserve or a provisional range estimate from being presented as a measured confidence interval.

For an illustrative 100 km leg with direct battery-side consumption of 18 kWh/100 km and known usable capacity of 60 kWh, nominal energy is 18 kWh and nominal SoC use is 30 percentage points. A departure at 80% predicts 50% arrival. This is an arithmetic example of the contract rather than a road-trial observation.

Table 3.3: Canonical energy and optimisation contract

Item	Latest development contract	Interpretation
Direct consumption	Distance × consumption / 100; usable capacity needed for SoC	Preserve provenance and battery-side measurement basis.
Rated range	Distance / rated range × 100; provisional	Preview-only for automatic charging-stop application; no fabricated consumption.
Battery state	Explicit initial SoC, chronological predictions and optional observations	Observations affect subsequent state without rewriting previous prediction history.
Reserve and uncertainty	Default reserve 12%; uncertainty uncalibrated	No arbitrary energy multiplier or measured confidence claim.
Charging	Compatible connector, effective power and saved target/duration	Unknown capacity or power produces partial/unavailable information.
Optimisation eligibility	Saved trip snapshot and provenance rules	Reject unsuitable apply requests on the server, not merely in the UI.

Item	Latest development contract	Interpretation
Consistency	Autosave flush, saved version and canonical capability checks	Older services cannot silently ignore trip-energy context.

For applying automatic stops, the trip service requires a suitable direct-consumption model and usable capacity. A legacy selection requires explicit confirmation; a user override must have the appropriate observed source; a catalogue default must meet the recorded provenance and battery-side source conditions. The server ignores request-supplied replacement vehicle and starting SoC values in favour of the owned saved trip.

Earlier days are replayed through road routing and canonical energy before optimising the selected day. Unknown or infeasible preceding legs cannot be certified merely because a later observation exists, and disconnected day boundaries are rejected. First/last charger endpoint protection, stale-state guards and saved charging context were implemented in development Phase 4A. All-leg reserve/detour edge cases and target/duration precision and application parity remain Phase 4B/4C work. These development phase labels are separate from this report's chapter numbering.

3.4.6 Security, Integrity and Failure Handling

Gateway authentication is only the first boundary. Private services validate ownership for trip, vehicle and account operations. Public publication uses an allowlist of approved content and does not expose the owner's account identifier. Selected public stops can carry valid location coordinates, while personal day-anchor information remains restricted. Revocation must remove public access rather than merely hiding an Explore card.

Optimistic version information protects against publishing an unexpected private version. Backend DTO validation constrains invalid coordinates, battery percentages and malformed commands. External URLs and media are subject to service validation. Database changes must be additive and compatible with rollback because a failed application release can leave its new schema in place.

3.4.7 Interface Design Principles

The planner is organised around a persistent itinerary context and a map panel. Public shared plans reuse planner components in a read-only state. This reduces divergence between the owner's and reader's representation, but editing controls must consistently honour the read-only condition. The design requires accessible labels, keyboard support, clear empty states and useful failure feedback. Screenshot appearance alone cannot establish those qualities; they require the evaluation in Chapter 4.

CHAPTER 4
SYSTEM FUNCTIONALITY

4.1 Introduction

This chapter describes the implemented modules, the test plan and the available results. It follows the capstone requirement to separate functionality from testing evidence [4]. Evidence has three levels: behaviour inspected in source code, automated results available in test artifacts, and integrated acceptance of a running system. A function being present in source does not automatically prove that every production workflow succeeds.

4.1.1 Accounts, Profiles and Vehicles

Sign-in uses Auth.js and Keycloak. User Management supports Navio profiles, personal vehicles, saved places and profile pictures. A managed vehicle catalogue supplies published model definitions and administrator-only catalogue actions. Guests can use permitted public planning data, while account writes and persisted private resources require authentication.

An account's saved vehicle definition is distinct from the route and battery assumptions used while preparing a trip. The latest development version includes a provenance-aware canonical energy profile, trip-specific garage membership and immutable vehicle-energy snapshots. User overrides, catalogue defaults and confirmed legacy values have different eligibility rules. The report follows the development revisions pinned by root dev, while retaining a distinction from the older production release.

4.1.2 Base Planner and Map Functions

The planner supports trip setup, itinerary blocks and items, notes, checklists and budgets/expenses. The map displays selected stops and available route geometry. Autosave communicates planner state to the trip service. Date changes, stop ordering and day start/end anchors must be reflected in the saved snapshot.

Place and route requests pass through service adapters. Provider data can contain useful location details, photos, distance and duration, but provider failure is still a possible operating condition. The interface should retain the user's planning context and show an actionable error when the route cannot be calculated.

4.1.3 EV Discovery and Planning

The selected vehicle controls compatibility filtering. Nearby charger discovery returns available provider data, and a charger can be inserted into a plan. EV route optimisation uses the service's planning logic and vehicle constraints. Energy and charging estimates follow the canonical contract described in Section 3.4.5. Before optimisation, the client flushes autosave, checks the saved version and requires the canonical-optimization-v1 capability. The trip service resolves the saved vehicle and initial SoC rather than accepting replacement energy inputs from the request.

The distinction between discovery and verification is essential. A compatible connector is a necessary input to a suggested stop; it is not a guarantee that the charger will be operating on arrival. Full community charger reviews and verification remain target-design functions in the inspected version.

4.1.4 Publication, Explore and Copying

An owner can preview and publish selected itinerary information, update publication, control Explore listing and revoke sharing. Explore uses a public feed of listed publications. Shared plans can display an approved title and author byline and render through read-only planner components. Trending uses stored view-count information.

A signed-in reader can request a separate private copy of a valid shared publication. The intended acceptance condition is independence: modifying the copied itinerary must not change the original or its public snapshot. Source and service tests establish parts of this contract, while a live two-account browser exercise remains an outstanding acceptance requirement in the release records.

4.1.5 Community and Administration

Community supports group pages, membership and group settings, post creation, threaded comments and voting through real API paths. Some auxiliary copied-trip references still use mock data, so the report does not treat every community widget as a completed shared-plan integration. Account administration and the global vehicle catalogue add role-gated operational functions.

The privileged-role workflow requires an appropriate actor and a reason for audited actions. Administrators can manage moderator assignments; OWNER supports administrator management, while OWNER assignment is retained as an identity-operator responsibility. Live role-grant acceptance with distinct accounts remains a follow-up rather than a newly measured result.

4.2 System Architecture

4.2.1 Implementation Modules

The current module arrangement is shown in Figure 3.2. The client is a separate repository, and the server contains four domain-service repositories plus the gateway, discovery and configuration directories. The root repository holds deployment configuration and pins child revisions. This makes version alignment an operational requirement: a correct service implementation can still be unreachable if the deployed gateway configuration omits its route.

Table 4.1: Current implementation modules and representative interfaces

Module	Responsibility	Representative interface
User Management	Profiles, garage, catalogue and administration	/v1/users/me; /v1/vehicle-models; /v1/admin/users
Trip Planning	Trips, planner snapshots, budget and publication	/v1/trips; /v1/trips/{id}/planner; publication routes
Mobility and EV	Places, directions, nearby chargers and optimisation	/v1/geo/places; /v1/routes/directions; /v1/ev/chargers/near
Community	Groups, posts, comments, votes and media	/v1/groups; /v1/posts; post comment and vote routes

Module	Responsibility	Representative interface
Platform applications	Routing, configuration and discovery	Gateway, Config Server and Eureka

4.2.2 Persistence and Deployment

Docker Compose configures the inspected production services and supporting PostgreSQL/PostGIS, Keycloak, Kafka and observability. Flyway runs before application schema validation. Production uses an externally mounted gateway configuration as well as configuration bundled in a service image. Route changes must therefore be checked in both relevant locations.

The report records the local inspected version and the dated release evidence. It does not infer the state of every production service on 7 October from an earlier deployment log. PostgreSQL upgrade checks, identity setup and live acceptance should be repeated for the actual final demonstration build.

4.2.3 Change from the Earlier Chapter 3 Design

The earlier report showed Clerk, Vault, a combined IAM/trip service and broader independent database/notification/AI structures. The implementation inspected here instead uses Keycloak, separate user and trip services, a shared PostgreSQL instance with service-owned schemas, and four implemented domain services. Retaining the original ER diagrams makes the design history available while Table 3.2 and this chapter identify the implemented subset.

4.3 Test Plan

4.3.1 Test Objectives and Environment

Testing should establish correct data handling, protected access, persistence, publication privacy, independent copying and consistent EV calculations. Automated checks should be supplemented by integrated browser acceptance and PostgreSQL tests. Test records should identify the client and service revisions, provider configuration, test accounts, browser and database version, execution date, expected outcome and observed result.

The report preparation reran the fifteen existing frontend test files from the latest development snapshot under Node.js 25.8.1 on Windows on 7 October 2026. The repository does not define a single test script in package.json. Backend results in Section 4.4 are read from existing Maven Surefire XML and are dated separately; those suites were not rerun during report preparation.

4.3.2 Functional and Access-Control Scenarios

Table 4.2: Functional and access-control acceptance scenarios

ID	Procedure	Expected result
TP-01	Sign in, refresh a protected page, expire the session and retry a write.	Identity is stable; expired sessions fail safely; refresh does not consume tokens incorrectly.

ID	Procedure	Expected result
TP-02	Create a two-day trip, reorder stops, add a note/checklist/expense, save and reload.	The owner recovers the saved values and order without unrelated-trip changes.
TP-03	As a second account, request the first account's private trip and vehicle mutations.	The owning service rejects access and makes no unauthorised change.
TP-04	Search places and compute a route; repeat with a timeout or invalid response.	Real provider data is used on success; failure is visible without fabricated geometry.
TP-05	Select vehicles with different connectors and toggle compatible-chargers filtering.	Results follow the selected vehicle and stale incompatible previews disappear.
TP-06	Evaluate zero/invalid distances, incompatible power and charge targets at 0%, 100% and unavailable-power boundaries.	Invalid inputs are rejected; valid results follow the canonical model, input eligibility and unavailable-data rules.
TP-07	Preview publication with optional dates, notes and budget; inspect its public JSON.	Only approved public data appears; private account and personal-anchor data remain absent.
TP-08	Publish, edit the private trip, update publication, then revoke the link.	Private edits wait for explicit publication update; revoked public access is unavailable.
TP-09	Copy a shared plan as a second signed-in user, edit it and reload both plans.	Copies persist independently; source content is unchanged; anonymous copying is rejected.
TP-10	Use posts/comments/votes and attempt another author's restricted edits.	Permitted actions persist; ownership and membership rules reject forbidden writes.
TP-11	Attempt catalogue and role changes as member, administrator and OWNER.	Only the appropriate actor succeeds; required audit reasons are preserved.
TP-12	Upgrade an existing PostgreSQL database and start every service.	Flyway history and entity validation succeed without losing pre-existing data.

4.3.3 Integration, Usability and Performance Evaluation

Integrated acceptance should exercise the gateway, identity provider and database together rather than using only stubbed APIs. Two distinct accounts are required for ownership and copy tests. Mobile-width layouts, keyboard-only operation and loading/error recovery require direct browser checks. Provider credentials and test fixtures must be recorded without placing secrets in the report.

A later usability study should ask participants to create an itinerary, insert a compatible charger and interpret the energy estimate. Measures can include task completion, observed errors and explanations of the estimate's assumptions. No participant results are reported here. A load test should similarly record the request mix, cache condition, concurrency, latency and error rate; the earlier capacity target is not an achieved measurement.

4.3.4 Evidence Recording and Completion Criteria

A test is complete only when its expected outcome and observed result are recorded. Source inspection can support the existence of an implementation, but cannot replace a live acceptance result. Skipped tests must be reported separately from executed passes. Any discrepancy between client, service and schema versions should be resolved before the final evaluation snapshot is approved.

4.4 Test Results

4.4.1 Frontend Automated Results Recorded for This Report

The report preparation reran all fifteen existing frontend *.test.mjs files from client development revision ba01a1b in an isolated temporary snapshot. The result was 119 passes, zero failures and zero skips under Node.js 25.8.1 on 7 October 2026. Shared Java/TypeScript golden fixtures were taken from the matching mobility revision b5a80eb. The snapshot reused the installed dependencies and did not change the user's application worktrees.

The runner used a temporary JSON import-attribute compatibility hook. An initial test of the older checked-out worktree had shown a standalone Node import issue; that older 57-check result is not presented as the latest development result. The isolated export also needed its root auth.ts file before the complete current run could load. The final 119-check run includes the complete required source files and unchanged assertions.

Table 4.3: Frontend checks rerun on 7 October 2026 against ba01a1b

Area	Test files or directory	Covered behaviour
Administration	tests/admin/admin.test.mjs	Roles, permitted proxies, audit reasons and moderation errors.
Authentication	tests/auth-session.test.mjs	Stable identity, token refresh and authenticated request handling.
Community	tests/community: three test files	Response parsing, post actions and safe upload proxy behaviour.

Area	Test files or directory	Covered behaviour
Garage	tests/garage: four test files	Vehicle API, catalogue flow, provenance-aware profiles and selection policy.
Energy and optimisation	tests/energy: four test files	Golden fixture parity, chronological state, autosave/version guards and delayed apply handling.
Guest planner	tests/guest-planner.test.mjs	Anonymous planning boundaries and protected persisted writes.
Charger filter	tests/planner/charger-filter.test.mjs	Connector filtering and stale-preview clearing.

This establishes the tested behaviours in the latest development snapshot. It is not a full distributed-service acceptance result, a new production deployment or physical EV prediction validation. Appendix A records the runner and source revisions.

4.4.2 Available Backend Automated Artifacts

Existing Surefire XML provides the results below. Counts are the sum of the retained per-class XML files in each service's target/surefire-reports directory. These records can include different execution times, so they are described as available artifacts rather than one freshly executed full-system run [16]. Failures and errors are both zero in the inspected XML; skips are shown separately.

Table 4.4: Retained backend test artifacts and their dates

Service / artifact date	Tests recorded	Executed passes / skips
User Management — 27 September 2026	147	145 passes; 2 skips; 0 failures or errors.
Trip Planning — 27 September 2026	157	157 passes; 0 skips; 0 failures or errors.
Mobility and EV — 26 September 2026	34	33 passes; 1 skip; 0 failures or errors.
Community — 14 September 2026	74	60 passes; 14 skips; 0 failures or errors.
API Gateway — 27 September 2026	12	11 passes; 1 skip; 0 failures or errors.

The retained results cover service behaviour represented by their test classes, including publication sanitisation, trip operations, gateway rules and EV calculation/provider cases. They must not be combined into an undated all-system pass claim. In particular,

the older Community artifacts do not establish that all newer integrated community workflows were tested against the present client.

4.4.3 Latest Development Verification and Publication

The updated project handoff records completion of development Phase 4A on 3 October and publication to dev on 5 October 2026. It records 175 Trip Planning tests and 39 Mobility tests passing with no failures, errors or skips, including disposable PostgreSQL schema/upgrade checks. It also records 119 frontend checks, TypeScript, targeted lint, a production build and synthetic guest/signed-in browser walkthroughs passing. These are dated team verification records; the backend suites and browser walks were not rerun during this report task [16].

The report's fresh 119-check frontend result agrees with that recorded unit-test total. The published revisions are client ba01a1b, trip 957cfbf, mobility b5a80eb and server f4a2813, pinned by the latest root dev snapshot. The release records state that production main was unchanged by that publication. Source publication is therefore distinguished from deployment or connected-service acceptance.

4.4.4 Dated Production Evidence and Outstanding Acceptance

The 27 September release log records successful deployment smoke checks: health and public shared-plan routes returned 200, while anonymous trip, copy and administration operations returned 401. It also records PostgreSQL checks for that earlier release and the addition of the shared-plan copy route. These are historical operational observations, not measurements of the newly published canonical optimisation version [16].

Live connected-service preview/apply/reopen acceptance and real-account role/copy checks remain required. The 3 October browser verification used synthetic APIs and cannot certify live charger-provider behaviour. Development Phase 4B must still address adversarial all-leg feasibility and reserve/detour cases; Phase 4C must complete charging precision and end-to-end preview/apply/reload parity. The report does not mark those work packages complete.

4.4.5 Evaluation Conclusions

The available evidence supports a working implementation of important planner, publication, community API, account and calculation behaviours. The fresh 119-check frontend run passes against the latest development snapshot under the documented runner arrangement. Latest Phase 4A backend and synthetic-browser results are recorded separately from older retained XML and production smoke checks. Historical backend artifacts show no recorded failures in their executed cases. However, complete integrated acceptance, load capacity, user usefulness and physical EV prediction accuracy remain unmeasured in the supplied evidence.

Charger feedback persistence, a complete notification experience and optional AI assistance should remain visible as further-development items. The project can be assessed fairly by separating those extensions from the main implemented workflows and by completing the missing acceptance records before making a final system-wide completion claim.

CHAPTER 5
SUMMARY AND SUGGESTIONS

5.1 Project Summary

Navio was developed to combine structured travel planning with EV-related information in a browser-based application. The problem carried forward from Chapters 1–3 was the coordination of itinerary organisation, routes, charging decisions, reuse and travel discussion. The current system contains a Next.js client, four Spring Boot domain services, an authentication boundary through Keycloak and gateway, PostgreSQL/PostGIS persistence and container-based supporting infrastructure.

The principal implementation includes private trip and planner state, notes/checklists/budget organisation, places and route integration, saved and catalogue vehicles, charger compatibility and planning logic, sanitised publication, Explore and independent-copy APIs. Community groups, posts, comments and voting provide a discussion channel. Administration supports account and catalogue operations under explicit roles. The original six ER diagrams remain part of the design record, with their implemented subset explained in Chapter 3.

5.1.1 Results Against the Project Objectives

Table 5.1: Objectives, available evidence and remaining work

Objective	Result supported by this version	Remaining evaluation or development
Structured trip planning	Planner and trip persistence are present with supporting API tests.	Complete saved-trip browser acceptance, responsive checks and multi-account access scenarios.
EV integration	Vehicle selection, charger queries, compatibility and a canonical energy and saved-trip optimisation contract are implemented.	Complete Phase 4B/4C and connected-service acceptance; develop charger feedback persistence; evaluate accuracy independently.
Plan sharing and reuse	Snapshot publication, listing, revocation and independent-copy contract are implemented.	Record a live two-account publish/copy/edit/revoke exercise.
Community discussion	Real group, post, comment and vote APIs and client workflows are present.	Recheck integrated permissions and replace remaining auxiliary mock trip references.
Secure, reproducible operation	Gateway/service controls, migrations and dated automated/release evidence are available.	Repeat final PostgreSQL startup and identity/role acceptance for the demonstration version.

Objective	Result supported by this version	Remaining evaluation or development
Conditional AI assistance	An optional design and AI ER diagram are retained.	Implement and evaluate the service only after the main workflows are stable.

5.1.2 Assessment of Completion and Benefits

The fresh frontend run records 119 passes against the latest development snapshot. Retained backend artifacts and dated release checks support additional implemented behaviours, with skipped and older cases identified separately. These results demonstrate technical progress toward the principal objectives. They do not prove that every planned entity, user workflow or production configuration is complete.

The expected benefits remain a coherent itinerary/charging workflow, reusable plans and structured community advice. No controlled user study or road-trial dataset was supplied, so the project does not report a measured reduction in planning time, anxiety or cost. This distinction allows the final conclusion to remain grounded in the actual work and its evidence.

5.2 Problems Encountered and Solutions

5.2.1 Divergence Between the Design and Implementation

The earlier design named authentication and infrastructure components that differed from later code. It also proposed more data entities than the migrations currently create. Using that design as an implementation inventory would produce misleading documentation and complicate integration. The response in this latest report is to update the architecture to Keycloak and four domain services, retain the original ER diagrams as target designs and provide an explicit persistence comparison.

For future iterations, one versioned architecture record should identify both intended and implemented capabilities. Each milestone should update the report, API contracts and migration inventory together. A feature should be described as completed only when its implementation and acceptance evidence refer to a compatible version.

5.2.2 Inconsistent Routing Across Deployment Configurations

The release records show a concrete routing issue: endpoints added to a service or development gateway configuration were missing from the mounted production gateway file. The backend could therefore contain the feature while public requests returned 404. The recorded solution added the missing routes to the deployed configuration and introduced route-oriented smoke checks.

The lesson is that configuration is part of the implementation contract. New endpoint acceptance should exercise the route used by the deployed browser, including anonymous and authenticated cases. Checking only a direct service request or a compiled image does not demonstrate that the production edge and gateway expose the intended behaviour.

5.2.3 Database Migration Compatibility During Parallel Work

The handoff records identify a migration-number collision between separate trip publication and energy/garage work. Once a migration has been applied or merged, editing its identity risks an inconsistent history. The required response is to preserve published migrations and allocate later versions to new changes, then test upgrade and application validation against PostgreSQL.

The inspected report version does not assume that every parallel branch has already been merged or deployed. Prevention requires agreeing migration ownership and the next version before parallel schema work, checking the existing history and retaining backward compatibility for rollback. H2 tests should not be substituted for that PostgreSQL startup check.

5.2.4 EV Input Quality and Estimate Interpretation

Vehicle inputs can differ in source and measurement basis. Declared capacity, usable capacity, rated range and direct battery-side consumption are not interchangeable. An earlier generic simulation margin would also obscure the distinction between nominal energy, reserve and uncertainty if copied into the new canonical model.

The implemented response is provenance-aware vehicle profiles, explicit model selection and a canonical nominal calculation. Rated-range fallback stays provisional; missing usable capacity is not fabricated. Initial SoC, observations and immutable vehicle snapshots belong to trip state. Shared golden fixtures compare Java and TypeScript behaviour, and the server applies eligibility rules to saved inputs before allowing automatic charging-stop application.

This establishes consistent semantics rather than physical accuracy. Uncertainty remains uncalibrated. The remaining Phase 4B/4C work includes route/reserve edge cases, precision and preview/application parity, followed by independent observations and user evaluation. A user observation can update subsequent state without making a previously infeasible leg a successful prediction.

5.2.5 Private Data Versus Public Plan Reuse

Sharing a live private trip could expose later edits or information beyond what the owner intended. The publication design addresses this through a server-sanitised frozen snapshot, opt-in fields and separate publication updates. Independent copying protects the source trip from the receiving user's changes. Revocation controls public access.

The remaining work is to verify the complete lifecycle with real accounts and the deployed gateway. Any new planner control or published field should be checked for the read-only view and public allowlist. Reusing planner components reduces duplicated interface code, but it also requires consistent enforcement of the public view's editing restrictions.

5.2.6 Testing Environment and Evidence Management

The first checked-out-worktree test encountered a standalone Node module-loading problem in the charger-filter test. The final report verification used the latest development snapshot and recorded all 119 checks passing. The project's TypeScript JSON import works in its web build context, while Node 25 required a JSON import attribute. A temporary runner hook supplied that attribute and the complete existing frontend set passed. No application source or assertion was changed.

The older retained backend artifacts had different execution dates and skipped cases, while newer Phase 4A verification was available in the project handoff. Recording dates, environment, skips and the tested revision prevents those artifacts from being presented as a single current acceptance result. A repeatable project test command and an evidence index would make final evaluation easier to reproduce.

5.3 Suggestions for Further Development

5.3.1 Complete Integrated Acceptance First

The first priority is a recorded end-to-end exercise with the final deployed client, gateway, services, Keycloak and PostgreSQL. Two accounts should demonstrate private ownership, publication updates, revocation and independent copies. Role changes should be checked with the required privileged actor. The team should record the actual revisions and observed outcomes for the Chapter 4 scenarios.

This evaluation should include failure recovery: expired sessions, provider timeouts, invalid public tokens and interrupted saves. It should also confirm that guest planning and public reads remain available while private writes are protected. These checks directly address the current evidence gaps without changing the principal project scope.

5.3.2 Improve EV Data and Validation

Develop a persisted charger domain with explicit source, freshness and verification information. Reviews and reports should have ownership rules and moderation workflows, and a report should not automatically become an operational truth. Provider quotas, timeouts and unavailable data should have predictable handling.

Complete the recorded Phase 4B/4C route and charging checks, then validate the canonical model against independent journeys and charging sessions. Store the input basis needed to interpret vehicle numbers and preserve the distinction between nominal energy and planning allowances. Maintain shared golden-fixture parity and add charging-target/duration precision cases and integrated checks for route detours, unknown station power and incompatible connectors. More detailed charging curves should follow trustworthy data rather than unsupported assumptions.

5.3.3 Evaluate Usability, Accessibility and Capacity

Run a bounded user evaluation with clearly defined planning tasks and informed participant records. Assess whether travellers can understand a charging suggestion and distinguish an estimate from a guaranteed outcome. Combine those observations with keyboard-only and responsive checks, including save/error recovery on smaller screens.

Measure performance under a documented workload, identifying the provider and cache conditions. Report latency, error rate and resource consumption instead of assigning an unsupported concurrent-user capacity. Observability dashboards and trace links should be validated against the actual deployment, especially where declared container limits approach available memory.

5.3.4 Complete Supporting Community and Notification Work

Replace auxiliary mock trip references with stable shared-plan contracts and add integrated community permission tests. Reconcile the wider target ERD with implemented media and moderation ownership. A notification module should handle

duplicate events and user preferences and should reveal only information the recipient may access.

Search, richer plan discussions and reporting can be added incrementally once their data and permission rules are agreed. These extensions should retain independence between the social conversation and the private itinerary being discussed.

5.3.5 Add AI Assistance as a Controlled Extension

If time and resources permit after the main system is stable, implement AI-assisted drafts or suggestions through the optional service boundary. Generated content should be validated against the same trip and mobility contracts, and a user should confirm a proposal before it changes a plan. Quotas and usage records in the retained target AI design can support cost and operational control.

AI should not invent charger availability or replace deterministic ownership and EV validation. Its usefulness should be evaluated using draft correctness, edit effort and user understanding. The optional extension should complement the completed planner rather than become a condition for demonstrating the principal project objectives.

5.3.6 Keep the Report and Release Reproducible

Maintain a release-linked report revision with updated diagrams, schema inventory, test records and known limitations. The original Chapter 3 figures should eventually be reconciled with the final implemented schema, while their historical originals remain preserved. A reproducible evaluation package should include scenario steps and evidence locations without credentials or personal data.

The project provides an integrated foundation for EV-aware itinerary planning, public reuse and travel discussion. Its final academic assessment should be based on the demonstrated main workflows, dated verification evidence and clearly identified further work.

REFERENCES

[1] International Energy Agency, 2025, Global EV Outlook 2025: Expanding Sales in Diverse Markets, IEA, Paris. Available: https://www.iea.org/reports/global-ev-outlook-2025 (accessed 7 October 2026).

[2] S. Upariphutthiphong, P. Thangsathityangkul and S. Duangsri, 2026, Navio: EV Trip Planner Platform (Thailand Focused), Chapter 3 report and preliminary chapter materials, unpublished CSC498 project documents, School of Information Technology, King Mongkut's University of Technology Thonburi. Source files: csc498-Navio Chapter 3.docx; csc498-Navio Chapter 1-2.docx; csc498-Navio Pre 4-5.docx.

[3] Google, n.d., Compute Routes Overview, Google Maps Platform Routes API documentation. Available: https://developers.google.com/maps/documentation/routes/compute-route-over (accessed 7 October 2026).

[4] School of Information Technology, King Mongkut's University of Technology Thonburi, n.d., Outline for CS Capstone Project Document, CSC498–CSC499 course guidance, supplied PDF: CSC498-CSC499 capstone-project-document (2).pdf.

[5] King Mongkut's University of Technology Thonburi, n.d., Thesis Manual (English Version) and Thesis_ENGVersion Word Templates, supplied CSC498-CSC499-project-doc-template folder. Formatting and native templates used for this report; the capstone outline determines chapter titles and required sections.

[6] Navio Project Team, 2026, Navio Application Source and Deployment Configuration, project repositories inspected 7 October 2026. Root: https://github.com/bestprappy/Navio; associated client and server submodules. Evidence includes client/package.json, service pom.xml files, controllers, migrations, simulation resources and .deploy/compose.production.yml.

[7] Google, n.d., Places API (New): Overview, Google Maps Platform documentation. Available: https://developers.google.com/maps/documentation/places/web-service/overview (accessed 7 October 2026).

[8] Wanderlog, n.d., Wanderlog Travel Planner, official product website. Available: https://wanderlog.com/ (accessed 7 October 2026).
[9] Vercel, n.d., Next.js Documentation: App Router, official documentation. Available: https://nextjs.org/docs/app (accessed 7 October 2026).
[10] Spring, n.d., Spring Boot Reference Documentation, official documentation. Available: https://docs.spring.io/spring-boot/index.html (accessed 7 October 2026).
[11] Keycloak, n.d., Securing Applications and Services with OpenID Connect, official documentation. Available: https://www.keycloak.org/securing-apps/oidc-layers (accessed 7 October 2026).
[12] PostgreSQL Global Development Group, n.d., PostgreSQL 16 Documentation: Schemas, Section 5.9. Available: https://www.postgresql.org/docs/16/ddl-schemas.html (accessed 7 October 2026).
[13] PostGIS Project, n.d., PostGIS Documentation, official documentation. Available: https://postgis.net/documentation/ (accessed 7 October 2026).
[14] Redgate, n.d., Versioned Migrations, Flyway product documentation. Available: https://documentation.red-gate.com/flyway/flyway-concepts/migrations/versioned-migrations (accessed 7 October 2026).
[15] Apache Software Foundation, n.d., Apache Kafka Documentation, official documentation. Available: https://kafka.apache.org/documentation/ (accessed 7 October 2026).
[16] Navio Project Team, 2026, Automated Verification and Deployment Records, unpublished project evidence. Sources: retained Maven Surefire XML dated 14–27 September 2026; docs/agents/session-log.md release entries; existing frontend tests rerun 7 October 2026. Artifact scope and skips are stated in Section 4.4.

APPENDIX A
TEST AND SOURCE RECORD

A.1 Report Basis and Document Provenance

This report uses the supplied native cover, abstract, acknowledgements, contents, list and chapter/reference templates, the team's Chapter 3 report and the capstone outline. The senior example was used to interpret academic structure; its project-specific claims and results were not copied into Navio. The earlier stand-alone NAVIO_Report_Chapters_1_to_5.docx was reviewed as background. This report takes its authoritative structure from the supplied templates and Chapter 3 document, and its current implementation claims from the latest development revisions rather than assuming that any earlier report is fully current.

The cover and approval details are carried from the team's Word report, with the author names and student IDs confirmed by the authors during preparation: Suriya Upariphutthiphong, 66130500858; Prapangkorn Thangsathityangkul, 66130500848; Supakrit Duangsri, 66130500856. Committee details are retained from that report. Six ERD images are reused as native Word images. They are labelled as target designs and are compared with actual migrations.

A.2 Frontend Test Reproduction

The existing tests use Node's built-in test runner and experimental TypeScript transformation. The successful report-preparation run used Node.js 25.8.1, all fifteen *.test.mjs files at development revision ba01a1b, a matching mobility golden-fixture export at b5a80eb, and a temporary module hook that supplied type: json for JSON import resolution. The initial older-worktree module-loading issue and the successful latest-snapshot run are distinguished in Section 4.4.1. The helper does not change assertions.

```
node --experimental-transform-types --import file:///path/to/json-import.mjs
--test <the fifteen existing test file paths>
```

The temporary helper imports registerHooks from node:module. Its resolve hook calls the next resolver and, only for a URL ending in .json, returns the result with importAttributes containing type: json. Other module resolutions are unchanged. The resulting runner summary was 119 tests, 119 passes, zero failures, zero skips and zero cancellations. This is not a substitute for an application build or browser acceptance.

A.3 Sample Integrated Acceptance Record

Table A.1: Acceptance record form for the publication and copy scenario

Record item	Required entry
Scenario	TP-09: independent copy of a public plan.
Environment	Final client/gateway/service revisions, PostgreSQL version, browser and date.
Accounts	Separate owner and reader test accounts; do not record credentials.

Record item	Required entry
Preparation	Owner publishes a known itinerary; reader can open its valid public link.
Steps	Reader signs in, copies, edits a stop and reloads both original and copy.
Expected result	Copy has a separate private identity; only the copied itinerary changes.
Observed result	Record actual behaviour, screenshots or request evidence after execution.
Current evidence state	Live two-account browser acceptance is not recorded in this report.

A.4 Backend Artifact Interpretation

The historical Table 4.4 totals are taken from TEST-*.xml files retained in each service's target/surefire-reports directory. Passing executions equal tests minus skips because failures and errors are zero in those inspected files. The dates are file-modification dates: User Management, Trip Planning and Gateway on 27 September; Mobility and EV on 26 September; Community on 14 September 2026. These are historical artifacts, not newly rerun suites.

The 27 September release narrative records operational smoke checks and PostgreSQL verification. Where that narrative and a retained XML total differ, this report uses the inspected artifact total for Table 4.4 and retains the narrative only as dated release evidence. The newer handoff records the separate Phase 4A 175-trip/39-mobility passing suites on 3 October; those records are reported separately in Section 4.4.3. The final team evaluation should archive a clean, revision-labelled run rather than rely indefinitely on retained local output.

A.5 Version Record

The latest root development snapshot inspected after repository sync is 9fb2619. It pins client ba01a1b and server f4a2813; the server pins Trip Planning 957cfbf, Mobility and EV b5a80eb, User Management 6183185 and Community 5f0cf41. The user's checked-out child worktrees were older than these pins. Current implementation claims were therefore checked through Git object reads and isolated temporary exports, without switching those worktrees or committing their existing local files.

The older retained Surefire artifacts and September production evidence relate to their own dates. The canonical changes were published to dev on 5 October and were not described by the handoff as a production deployment. This record prevents the report's latest implementation description from being confused with a fresh live release.

A.6 Code Evidence Map

Table A.2: Representative source locations for report claims

Claim	Representative source
Web stack and package versions	client/package.json; client/app and its feature modules.
Authentication and roles	client auth/session helpers; gateway GatewaySecurityConfig.java; IAM role services.
Planner persistence	Trip Planning controllers, planner DTOs and Flyway migration files.
Public snapshot and copying	TripPublicationService.java; PlanPublicationSanitizer.java; SharedPlanController.java.
EV estimate	CanonicalEnergy.java; client energy-model.ts; TripOptimizationEnergy.java; PrecedingTripEnergy.java; shared energy-v1 fixtures.
Community core	Community group, post and comment controllers; social migrations and community-api.ts.
Actual deployment configuration	.deploy/compose.production.yml; .deploy/config/api-gateway.yml.
Historic verification	Service target/surefire-reports XML; docs/agents/session-log.md.